RDMFT-for-HEG: Algorithm Review and Formula Summary

This document reviews the **complete algorithm** used to solve the RDMFT (Reduced Density Matrix Functional Theory) equations for the 3-D paramagnetic **homogeneous electron gas (HEG)** in this repository, checks each step for correctness, and summarises all formulas and equations actually implemented in the code.

Source map:

Concern	File
Geometry / HF analytic energies	`include/HEG.hpp`
k-grid and quadrature weights	`include/Grid.hpp`
Log-singular exchange quadrature (matrix W)	`include/ExchangeKernel.hpp`
Two-body kernels $K(n_i, n_j)$	`include/Functional.hpp`
Energy / density / gradient evaluators	`include/Energy.hpp`
Self-consistent solver	`include/Solver.hpp`
PW92 / Ortiz–Ballone references	`include/QMC.hpp`
CLI driver	`src/main.cpp`

All quantities are in **Hartree atomic units**. Occupations $n(k) \in [0, 1]$ are **per spin orbital** and depend only on $k = |\mathbf{k}|$ (plane-wave natural orbitals). The spin factor of 2 is folded into the prefactors below.

---

1. Physical model and analytic benchmarks

Wigner–Seitz radius r_s, density, Fermi wave-vector (`HEG.hpp`):

$$\rho = \frac{3}{4\pi r_s^3}, \qquad k_F = \frac{(9\pi/4)^{1/3}}{r_s}.$$

Hartree–Fock energy per electron (used only to define the RDMFT correlation energy as a difference):

$$t_s = \tfrac{3}{10} k_F^2, \qquad e_x = -\frac{3}{4\pi} k_F, \qquad e_{\mathrm{HF}} = t_s + e_x.$$

Check. Both expressions are the textbook paramagnetic HEG results. Consistency between $\rho = k_F^3/(3\pi^2)$ and $\rho = 3/(4\pi r_s^3)$ fixes $k_F = (9\pi/4)^{1/3}/r_s$, matching `HEG::kF`. ✔

---

2. Energy functional (per volume)

The evaluators in `Energy.hpp` implement

$$\frac{T}{V} = \frac{1}{2\pi^2}\int_0^\infty k^4\, n(k)\, dk,$$

$$\rho = \frac{1}{\pi^2}\int_0^\infty k^2\, n(k)\, dk,$$

$$\frac{E_{xc}}{V} = -\frac{1}{2\pi^3}\int_0^\infty\!\!\int_0^\infty k\,k'\,K\big(n(k),n(k')\big)\,\ln\left|\frac{k+k'}{k-k'}\right|\,dk\,dk'.$$

Derivation check of the prefactors. Starting from the spin-summed momentum-space exchange integral with Coulomb kernel $v(q) = 4\pi/q^2$,

$$\frac{E_x}{V} = -\tfrac{1}{2}\sum_\sigma \int \frac{d^3k}{(2\pi)^3}\frac{d^3k'}{(2\pi)^3} n(k)n(k')\frac{4\pi}{|\mathbf{k}-\mathbf{k}'|^2},$$

the angular integral gives $\int d\Omega_k\, d\Omega_{k'}\, |\mathbf{k}-\mathbf{k}'|^{-2} = (8\pi^2/kk')\ln|(k+k')/(k-k')|$, and collecting factors ($\tfrac{1}{2}\cdot 2\cdot 4\pi\cdot 8\pi^2/(2\pi)^6 = 1/2\pi^3$) reproduces the E_{xc}/V prefactor above with $K = nn'$ for HF. ✔ Likewise T/V and ρ reduce to the quoted $1/2\pi^2$ and $1/\pi^2$ prefactors. ✔

End-to-end analytic check. For the step occupation $n(k) = \theta(k_F - k)$ one finds $E_x/V = -k_F^4/(4\pi^3)$, hence $e_x = E_x/\rho = -3k_F/4\pi$, matching `HEG::exchange_per_electron_HF`. This is verified numerically by `tests/test_hf_exchange.cpp` (relative error $\lesssim 5\times10^{-3}$ at $N = 401$). ✔

---

3. Log-singular quadrature: the matrix W

The inner integral over k' contains the **integrable logarithmic singularity** at $k' = k_i$. `ExchangeKernel.hpp` uses **product integration**: on each grid interval $[k_s, k_{s+1}]$ the smooth factor is treated as piecewise linear (hat functions) and the log terms are integrated **analytically** against those hats. With the convention $0\ln 0 = 0$ and primitives

$$F_0(t) = \int \ln|t|\,dt = t\ln|t| - t, \qquad F_1(t) = \int t\ln|t|\,dt = \tfrac{1}{2}t^2\ln|t| - \tfrac{1}{4}t^2,$$

the interval moments of $P(k_i,k') = \ln(k_i+k') - \ln|k_i-k'|$ are

$$M_0 = \int_a^b P\,dk', \qquad M_1 = \int_a^b k'\,P\,dk',$$

and the two hat contributions add to the dense matrix W:

$$W_{i,s} \mathrel{+}= \frac{bM_0 - M_1}{h}, \qquad W_{i,s+1} \mathrel{+}= \frac{M_1 - aM_0}{h}, \qquad h = b-a,$$

so that the inner "exchange potential" is

$$V_i = \sum_j W_{ij}\,k_j\,(\cdots)_j, \qquad \frac{E_{xc}}{V} = -\frac{1}{2\pi^3}\sum_i w_i\,k_i\,V_i.$$

A transpose `Wt` ($\mathrm{Wt}_{ij} = W_{ji}$) is precomputed for cache-friendly access in the gradient.

Check. The substitutions $u = k' - c$ and $u = k_i + k'$ reduce $\int_a^b \ln|c-k'|dk'$, $\int_a^b k'\ln|c-k'|dk'$, $\int_a^b \ln(k_i+k')dk'$ and $\int_a^b k'\ln(k_i+k')dk'$ exactly to differences of F_0, F_1 as coded in `integ_logabs_c`,

`integ_kp_logabs_c`, `integ_logplus`, `integ_kp_logplus`. The hat weights $(bM_0 - M_1)/h$ and $(M_1 - aM_0)/h$ are the exact integrals of $\phi_0(k') = (b - k')/h$ and $\phi_1(k') = (k' - a)/h$ against P. ✔ Because the result is exact for piecewise-linear integrands, the HF step energy is reproduced to high accuracy on the production grid.

---

4. Two-body kernels $K(n_i, n_j)$

Default separable form $K(n_i, n_j) = f(n_i) f(n_j)$ (given by `f`,`df`); richer functionals override `kernel`/`kernel_grad`.

Functional	$f(n)$ or K	Notes
HF	$f(n) = n$	exact exchange
Müller / BB	$f(n) = \sqrt{n}$	$\alpha = \frac{1}{2}$
GU	$=$ Müller in HEG	plane-wave self-interaction terms vanish
Power(α)	$f(n) = n^\alpha$	Sharma et al.; $\alpha \approx 0.55$–0.58
CGA	$K = \frac{1}{2}[\, n_i n_j + \sqrt{n_i(2-n_i)}\sqrt{n_j(2-n_j)}\,]$	Csányi–Goedecker–Arias
CHF	$K = n_i n_j + \sqrt{n_i(1-n_i)n_j(1-n_j)}$	implemented as `BetaFunctional(1/2)`
Beta(β)	$K = n_i n_j + [\, n_i(1-n_i)n_j(1-n_j)\,]^\beta$	$\beta = \frac{1}{2} \Rightarrow$ CHF; $\beta \to \infty \Rightarrow$ HF
GEO	see §4.1	3-channel, non-separable
OptGeo	$K = n_i n_j + w\,[2\sigma(k\,x_{ij}) - 1]$, $x_{ij} = (n_i - \frac{1}{2})^2(n_j - \frac{1}{2})^2$	HF + sigmoid correction
HybOpt	$(1-\lambda) n_i n_j + \lambda\, n_i^\alpha n_j^\alpha$	$\lambda \in [0,1]$, $\alpha \in (0,1)$
BOW	$K = x^\alpha - \alpha x + \alpha - \alpha(1-x)^{1/\alpha}$, $x = n_i n_j$	Baldsiefen et al.; default $\alpha = 0.61$
SymBow	$\frac{1}{2}[K_{\mathrm{BOW}}(n_i, n_j) + 1 - K_{\mathrm{BOW}}(1-n_i, 1-n_j)]$	particle-hole symmetrised BOW
BBC1	Müller with smoothed weak–weak sign flip	experimental
BBC3	weak–weak / strong–strong / rest rule (smoothed)	equals BBC2 in HEG

For every non-separable kernel, `kernel_grad(a,b)` must equal $\partial K / \partial a$. This was verified analytically term-by-term for CGA, Beta/CHF, OptGeo, HybOpt, BOW, SymBow, BBC1, BBC3 — all consistent. ✔ The single exception (GEO) is documented next.

4.1 GEO kernel — bug found and fixed

The GEO kernel is a normalised three-channel blend of HF, Müller and Power(3/4) with weights $1 : 1 : 2$ (so that $K_{\mathrm{GEO}}(1,1) = 1$), i.e. with $p = n_i n_j$:

$$\boxed{K_{\mathrm{GEO}}(n_i, n_j) = \tfrac{1}{4}\Big[\, p + p^{1/2} + 2\,p^{3/4} \Big] = \tfrac{1}{4}p + \tfrac{1}{4}p^{1/2} + \tfrac{1}{2}p^{3/4}}$$

with $K_{\mathrm{GEO}}(0, \cdot) = 0$. The channel derivatives are

$$\frac{\partial K_{\mathrm{GEO}}}{\partial n_i} = \tfrac{1}{4}\Big[\, n_j + \tfrac{1}{2}\sqrt{n_j/n_i} + \tfrac{3}{2}\, n_j\, (n_i n_j)^{-1/4} \Big].$$

The bug. The previous `GEOFunctional::kernel` computed

$$K^{(\mathrm{old,\ wrong})} = \tfrac{1}{4}\big(p + 2\,p^{1/2} + p^{3/4}\big),$$

i.e. it placed the factor 2 on the $p^{1/2}$ (Müller) channel instead of the $p^{3/4}$ (Power) channel — weights $(1, 2, 1)$ instead of $(1, 1, 2)$. However:

- `GEOFunctional::kernel_grad` used weights $(1, 1, 2)$;
- the solver's `update_occupations_geo` uses channel weights $(c_1, c_2, c_3) = (\tfrac{1}{4}, \tfrac{1}{4}, \tfrac{1}{2})$, i.e. $(1, 1, 2)$;
- the README and class docstring both state $1 : 1 : 2$.

So the **energy** was evaluated with a *different* kernel than the one used to find the **occupations** and the gradient. The inconsistency was masked because both conventions coincidentally satisfy $K(1, 1) = 1$. This is a genuine correctness bug: the reported GEO energy did not correspond to a stationary point of the functional it was solving.

The fix. `kernel` now returns `0.25*(p + s12 + 2*s34)`, matching `kernel_grad`, the solver and the documentation. Effect on the GEO correlation energy (401-point grid, $k_{\max} = 3k_F$):

r_s	E_c **before (wrong)**	E_c **after (fixed)**
1	-0.0318 Ha	-0.0168 Ha
5	-0.0153 Ha	-0.0077 Ha

`data/GEO.tsv` has been regenerated with the corrected kernel. No other functional was affected.

---

5. Euler–Lagrange equation

Minimising $T + E_{xc} - \mu\rho$ at fixed density, subject to $0 \le n_i \le 1$, gives (using $\partial(T/V)/\partial n_i = \frac{1}{2\pi^2} w_i k_i^4$ and $\partial\rho/\partial n_i = \frac{1}{\pi^2} w_i k_i^2$) the **quasiparticle energy** used by the projected-gradient branch (`pseudo_energy`):

$$\varepsilon_i = \frac{k_i^2}{2} + \frac{1}{\mathrm{pref}_i}\,\frac{\partial (E_{xc}/V)}{\partial n_i}, \qquad \mathrm{pref}_i = \frac{w_i k_i^2}{\pi^2},$$

with, for a symmetric kernel,

$$\frac{\partial (E_{xc}/V)}{\partial n_i} = -\frac{k_i}{\pi^3} \sum_j w_j\, k_j\, W_{ji}\, \frac{\partial K}{\partial n_i}(n_i, n_j).$$

The KKT/complementarity conditions are: $\varepsilon_i = \mu$ where $0 < n_i < 1$; $\varepsilon_i > \mu \Rightarrow n_i = 0$; $\varepsilon_i < \mu \Rightarrow n_i = 1$.

Check on the discretisation. The closed-form branches use $U_i = \sum_j W_{ij} k_j f(n_j)$ (row of W, no explicit w_j), whereas `deps_xc` uses $\sum_j w_j k_j W_{ji}(\cdots)$ (column of W, explicit w_j). Since the hat integral makes $W_{ij} \approx w_j\, P(k_i, k_j)$ and P is symmetric, one has $W_{ji} \approx w_i P(k_i, k_j)$, so $w_j k_j W_{ji}/w_i \approx W_{ij} k_j$: the two forms are the **same** discretisation of the continuum gradient $\delta E_{xc}/\delta n(k) = -\frac{k}{\pi^3} \int k' K_1 \, \ln|\cdots| \, dk'$, up to the intrinsic quadrature error (the exact discrete-energy gradient is their symmetric average). Both converge to the correct continuum EL, and the line-search branch tests the *actual* energy so it can never take an energy-increasing step. ✔

---

6. Self-consistent solver (`Solver.hpp`)

6.1 Chemical-potential bisection

$\rho(\mu)$ is monotonically increasing, so for each occupation-update rule the solver brackets and **bisects** μ until $\rho(\mu) \approx \rho_{\text{target}}$ (`adaptive_bisect`, default bracket $[-50, 50]$, geometrically expanded up to 16 times if the bracket is too tight). ✔ (monotonicity guarantees a unique root).

6.2 Occupation-update branches

1. **Power family** (HF, Müller, GU, Power): closed-form inversion of $\alpha n_i^{\alpha-1} U_i = \pi k_i (\frac{k_i^2}{2} - \mu)$ for $\alpha \neq 1$ (`update_occupations_power`), giving $n_i = \left[R/(\alpha U_i)\right]^{1/(\alpha-1)}$ clamped to $[0, 1]$. For HF ($\alpha \to 1$) the occupation is a step, smoothed at the single node nearest μ so $\rho(\mu)$ stays continuous for the outer bisection.
2. **Generic separable** kernels: 1-D bisection of $df(n_i)U_i = R$ (`update_occupations_factorized`).
3. **Additive** $K = a[n_i n_j + g(n_i)g(n_j)]$ (CGA $a = \frac{1}{2}$; Beta/CHF $a = 1$): solve $U_i^{HF} + g'(n_i)U_i^g = R/a$ with a dedicated 1-D inverter (CGA: $n = 1 - s/\sqrt{1+s^2}$; Beta: bracketed root of $u(1-u^2)^{\beta-1} = B$, $u = 1 - 2n$). ✔
4. **GEO / OptGeo channel form**: three channel sums $U_{1,2,3}$, LHS strictly decreasing in n_i (for $U_\alpha \geq 0$), solved by bisection on $(0, 1)$.
5. **HybOpt**: two-channel HF+Power, monotone LHS, bisection.
6. **Everything else** (BBC1, BBC3, OptGeo sigmoid, BOW, SymBow, NN-pair): **projected gradient** with **Armijo backtracking** line search ($n_i \leftarrow \mathrm{clip}[\, n_i - \mathrm{step}\,(\varepsilon_i - \mu)\,]$, step starting at 1 and halved), with an inner μ bisection per trial step so the density constraint is met before the energy decrease is tested.

6.3 Mixing, convergence, multi-start

Linear mixing $n_i \leftarrow (1-\eta) n_i + \eta\, n_i^{\text{target}}$ ($\eta =$ `mix`); convergence when $\max_i |\Delta n_i| <$ `tol_n`. Driver overrides: `tol_n=1e-8`, `max_iter=1200`, `mix=0.40`. For kernels with a competing HF-like minimum (additive, GEO/OptGeo, HybOpt, BBC3, BOW) the solver runs **several smeared initial guesses** and keeps the **lowest-energy** converged result (variational selection). ✔

---

7. Correlation energy and reference

The RDMFT correlation energy reported per electron is

$$E_c^{\text{RDMFT}} = \frac{E^{\text{RDMFT}}}{N} - \frac{E^{\text{HF}}}{N},$$

compared against the PW92 parameterisation (`QMC.hpp`):

$$e_c(r_s) = -2A(1+\alpha_1 r_s)\,\ln\!\left(1+\frac{1}{2A\left(\beta_1\sqrt{r_s}+\beta_2 r_s+\beta_3 r_s^{3/2}+\beta_4 r_s^2\right)}\right),$$

with $A = 0.0310907$, $\alpha_1 = 0.21370$, $\beta_1 = 7.5957$, $\beta_2 = 3.5876$, $\beta_3 = 1.6382$, $\beta_4 = 0.49294$ — the standard PW92 paramagnetic set. ✔

---

8. Summary of the audit

Component	Status
Density, kinetic, exchange prefactors	✔ correct (derived + numerically checked)
Log-singular product integration (W)	✔ correct (exact for piecewise-linear integrands)
Euler–Lagrange / `pseudo_energy`	✔ correct; two gradient discretisations consistent
μ bisection / density constraint	✔ correct (monotone, adaptive bracket)
Power / factorized / additive / HybOpt occupation updates	✔ correct closed-form / monotone inversions
Projected-gradient + Armijo line search	✔ energy-monotone, robust
`kernel_grad` vs `kernel` (CGA, Beta, CHF, OptGeo, HybOpt, BOW, SymBow, BBC1, BBC3)	✔ consistent
`kernel_grad` vs `kernel` (GEO)	✗ **fixed** — factor 2 was on the wrong channel
PW92 reference	✔ correct
Unit tests (`test_hf_exchange`, `test_gz_momentum`, `test_nn_functional`)	✔ pass after the fix

Conclusion. The RDMFT algorithm is sound: the energy/gradient discretisation, the log-singular quadrature, the Euler–Lagrange conditions, the μ-bisection density constraint, and the per-family occupation updates are all correct. The one defect was an **internal inconsistency in the GEO kernel** (energy evaluated with weights $1:2:1$ while the occupations/gradient used $1:1:2$); this has been corrected so that GEO's energy is now stationary for the distribution the solver produces.

Minor (non-bug) observations

- The two gradient discretisations (`compute_U` using `w_row` without w_j vs `deps_xc` using `wt_row` with w_j) agree only up to quadrature error, not bit-for-bit; this is expected and harmless because the closed-form branches don't use `deps_xc` and the PGD branch line-searches the true energy.

- `docs/program_summary_zh.md` previously documented the GEO kernel with the buggy $1:2:1$ weights (declaring the code authoritative); it has been updated to the corrected $1:1:2$ form.